

Estructura de Datos
Quiz #3
Título: Dynamic Arrays and Amortized Analysis

Diego Rodriguez
Universidad Nacional de Colombia Sede Bogotá, Facultad de Ingeniería.
Docente: Sebastián Yepes García
Semestre 2026-02
Fecha: 23 de septiembre, 2026

1. Dynamic Arrays From Scratch

1.1. Representación Interna de `DynamicArray<T>`

La estructura `DynamicArray<T>` es una implementación genérica de un arreglo de tamaño dinámico construida desde cero, sin utilizar estructuras de la biblioteca estándar de Java (como `ArrayList` o `Vector`). Su estado interno está definido por tres atributos privados:

- `private T[] data`: Arreglo nativo interno para almacenar elementos genéricos del tipo T .
- `private int size`: Número de elementos lógicos almacenados actualmente ($0 \leq \text{size} \leq \text{capacity}$).
- `private int capacity`: Capacidad física total reservada en memoria ($\text{capacity} \geq 1$).

Debido al borrado de tipos (*Type Erasure*) en Java, no es posible instanciar directamente un arreglo genérico (`new T[capacity]`). Por esta razón, el arreglo subyacente se crea mediante un casteo explícito: `(T[]) new Object[capacity]`.

Listing 1: Constructor e inicialización de `DynamicArray<T>`

```
1 public class DynamicArray<T> {
2     private T[] data;
3     private int size;
4     private int capacity;
5
6     @SuppressWarnings("unchecked")
7     public DynamicArray(int initialCapacity) {
8         if (initialCapacity < 1) {
9             throw new IllegalArgumentException("La capacidad inicial debe ser >= 1");
10        }
11        this.capacity = initialCapacity;
12        this.size = 0;
13        this.data = (T[]) new Object[initialCapacity];
14    }
15 }
```

1.2. Invariantes de Representación de `DynamicArray<T>`

Para garantizar el correcto funcionamiento de la estructura antes y después de cada operación pública (`get`, `set`, `append`, `removeLast`), se preservan formalmente los siguientes invariantes:

1. Invariante de Capacidad y Tamaño (Cota de Estado):

$$0 \leq \text{size} \leq \text{capacity} \quad \wedge \quad \text{capacity} \geq 1 \quad (1)$$

Demostración: El constructor asegura $\text{capacity} \geq 1$ y $\text{size} = 0$. Toda operación `append` incrementa `size` en 1; si $\text{size} = \text{capacity}$, se ejecuta un `resize(capacity * 2)` duplicando la capacidad antes de insertar el elemento. La operación `removeLast` decrementa `size` en 1 exigiendo que $\text{size} > 0$.

2. Invariante de Contigüidad Física de Memoria:

$$\forall i \in [0, \text{size} - 1], \quad \text{data}[i] \neq \text{null} \quad \wedge \quad \forall j \in [\text{size}, \text{capacity} - 1], \quad \text{data}[j] = \text{null} \quad (2)$$

Demostración: Los elementos lógicos ocupan de forma continua las celdas físicas desde el índice 0 hasta $\text{size} - 1$. La operación `removeLast` anula explícitamente la referencia en `data[size]` para permitir la recolección de basura (*Garbage Collection*) y evitar fugas de memoria.

3. Invariante de Preservación de Capacidad en Reducción:

Tras cualquier secuencia de llamadas a `removeLast()`, `capacity` no se reduce. (3)

Demostración: La especificación prohíbe explícitamente reducir la capacidad física al desapilar o eliminar elementos.

2. Snake Redesign Challenge

2.1. Representación Interna Circulante

Para resolver la ineficiencia de la representación ingenua (donde `removeTail` requería desplazar elementos con costo $\Theta(k)$), la clase `Snake` implementa un **arreglo nativo circular** administrado por un número constante de variables auxiliares:

- `private Object[] data`: Arreglo físico nativo reservado en memoria.
- `private int tail`: Índice físico en el arreglo que almacena el elemento en el índice lógico 0 (la cola / TAIL).
- `private int size`: Longitud lógica actual de la serpiente (k).
- `private int capacity`: Capacidad física total reservada en memoria (C).

2.2. Mapeo Modular Formal (Lógico $\rightarrow$ Físico)

La estructura preserva la convención del examen: el índice lógico 0 es la COLA (TAIL) y el índice lógico $size - 1$ es la CABEZA (HEAD). La función de mapeo que transforma un índice lógico $i \in [0, size - 1]$ en su posición física real en el arreglo es:

$$phys(i) = (tail + i) \pmod{capacity} \quad (4)$$

2.3. Invariantes de Representación de Snake

1. Invariante de Rango Modular:

$$0 \leq tail < capacity \quad \wedge \quad 0 \leq phys(i) < capacity \quad \forall i \in [0, size - 1] \quad (5)$$

2. Invariante de Continuidad Cíclica:

$$phys(i + 1) = (phys(i) + 1) \pmod{capacity} \quad \forall i \in [0, size - 2] \quad (6)$$

3. **Invariante sin desplazamiento:** Ningún movimiento ordinario `addHead` / `removeTail` desplaza físicamente los k elementos del cuerpo. La eliminación de la cola solo realiza $tail = (tail + 1) \pmod{capacity}$ con costo $\Theta(1)$.

3. Análisis de Complejidad Temporal y Bottlenecks

3.1. Demostración de Complejidades

3.1.1. Pregunta 1: `DynamicArray<T>`

- `get(index)` / `set(index, value)`: $\Theta(1)$ en el peor caso. Acceso directo por índice de memoria.
- `removeLast()`: $\Theta(1)$ en el peor caso. Anula la referencia y decrementa el puntero de tamaño sin alterar la capacidad.
- `append(value)`: $O(n)$ en el peor caso cuando $size = capacity$ (requiere copiado elemento por elemento), pero $O(1)$ amortizado.

3.1.2. Pregunta 2: Enfoque Ingenuo con Desplazamiento

- **Complejidad de `REMOVE_TAIL()`:** Al remover la cola ubicada en el índice 0, el algoritmo ejecuta un bucle `for` que desplaza físicamente $k - 1$ elementos hacia la izquierda:

$$T_{\text{removeTail}}(k) = \sum_{i=1}^{k-1} \Theta(1) = \Theta(k) \quad (7)$$

- **Complejidad de m movimientos sin comer:** Si una serpiente de longitud constante k realiza m movimientos sin comer, el costo total acumulado es:

$$T(m, k) = m \times (\Theta(1) + \Theta(k)) = \Theta(m \cdot k) \quad (8)$$

- **Falsedad de la afirmación del estudiante:** La afirmación *Como append() es $O(1)$ amortizado, entonces MOVE() es $O(1)$ amortizado* es **FALSA**. Aunque la inserción de la cabeza tome costo constante promedio, el movimiento completo requiere llamar obligatoriamente a REMOVE_TAIL(), el cual incurre en un costo no negociable de $\Theta(k)$ en cada paso.
- **Cuello de Botella (Bottleneck):** El desplazamiento físico secuencial de todos los elementos para mantener la cola fija en la posición 0.

3.1.3. Pregunta 3: Rediseño Circular

- **removeTail():** $\Theta(1)$ worst-case. Obtiene la posición física en `data[tail]`, la anula, incrementa la cola de forma modular $\text{tail} = (\text{tail} + 1) \pmod{\text{capacity}}$ y reduce `size-`. No realiza desplazamientos de datos.
- **addHead(p):** $O(1)$ amortizado. Calcula la casilla de inserción $\text{phys}(\text{size})$, asigna el valor e incrementa el tamaño. Si $\text{size} = \text{capacity}$, duplica la capacidad mediante desenrollado ordenado en $\Theta(\text{size})$.

4. Demostración Formal del Análisis Amortizado

4.1. Crecimiento Transitorio y Cota Superior

En cada movimiento ordinario, el juego ejecuta primero `addHead()` y luego `removeTail()`. Sea $g \leq m$ el número total de manzanas consumidas durante una secuencia de m movimientos. Durante un movimiento sin comer, la estructura incrementa transitoriamente su tamaño a $k + 1$ antes de desapilar la cola. Por ende, la longitud máxima transitoria que alcanza el arreglo es:

$$n_{\text{máx}} = 2 + g + 1 = g + 3 \quad (9)$$

4.2. Método Agregado (Aggregate Method)

Dado un estado inicial con $\text{size}_0 = 2$ y $\text{capacity}_0 = 2$, los eventos de redimensionamiento (`resize`) se activan al alcanzar capacidades $C = 2, 4, 8, 16, \dots, 2^j$ cuando $\text{size} = C + 1$.

El número total de elementos copiados en todas las operaciones de redimensionamiento es la suma de la serie geométrica de las capacidades previas:

$$\text{Copias Totales} = \sum_{j=1}^{\lfloor \log_2 n_{\text{máx}} \rfloor} 2^j = 2 + 4 + 8 + \dots + 2^k = 2^{k+1} - 2 < 2n_{\text{máx}} \quad (10)$$

Sustituyendo $n_{\text{máx}} = g + 3$:

$$\text{Copias Totales} < 2(g + 3) = O(g) \quad (11)$$

4.2.1. Costo Amortizado

- Costo real de escrituras directas en m movimientos: $O(m)$.
- Costo de duplicación por `resize`: $O(g)$.
- Costo Total de la secuencia de m movimientos: $T(m, g) = O(m + g) = O(m)$ (dado que $g \leq m$).
- **Costo Amortizado por operación `addHead()`:**

$$\hat{c} = \frac{T(m, g)}{m} = \frac{O(m)}{m} = O(1) \quad \text{constante} \quad (12)$$

4.3. Método del Potencial (Potential Method)

Definimos la función potencial para un estado S con tamaño k y capacidad C como:

$$\Phi(S) = 2k - C \quad (13)$$

4.3.1. Propiedades de la Función Potencial

1. **Potencial Inicial:** Con $\text{size} = 2$ y $\text{capacity} = 2$, $\Phi(S_0) = 2(2) - 2 = 2 \geq 0$.
2. **No Negatividad:** Como el redimensionamiento ocurre cuando $k = C$, en todo estado válido se cumple $C \leq 2k$, garantizando que $\Phi(S) \geq 0$.

4.3.2. Análisis de Casos para `addHead()`

El costo amortizado se define como $\hat{c}_i = c_i + \Phi(S_i) - \Phi(S_{i-1})$.

- **Caso A: Sin `resize` ($k < C$)**
 - Costo real: $c_i = 1$.
 - Cambio de potencial: $\Delta\Phi = (2(k + 1) - C) - (2k - C) = 2$.

- Costo amortizado: $\hat{c}_i = 1 + 2 = 3 = O(1)$.
- **Caso B: Con resize ($k = C$, la capacidad pasa de C a $2C$)**
 - Costo real: $c_i = C + 1$ (copiar C elementos e insertar la nueva cabeza).
 - Potencial anterior: $\Phi(S_{i-1}) = 2C - C = C$.
 - Potencial nuevo (con $k + 1$ elementos y capacidad $2C$): $\Phi(S_i) = 2(C + 1) - 2C = 2$.
 - Cambio de potencial: $\Delta\Phi = 2 - C$.
 - Costo amortizado:

$$\hat{c}_i = (C + 1) + (2 - C) = 3 = O(1) \quad (14)$$

Efecto de `removeTail()`: Cuando se ejecuta `removeTail()`, el tamaño decrece en 1 sin alterar la capacidad C , lo cual reduce el potencial en $\Delta\Phi = -2$. Esta liberación de energía paga por la eliminación sin afectar la validez de la cota $O(1)$.

5. The Adversarial Snake y Políticas de Crecimiento

5.1. Análisis Frente al Adversario

Un adversario controla el consumo de manzanas g . En ausencia de contracción de capacidad, el momento exacto en que ocurren los crecimientos no altera el total de elementos copiados acumulados hasta alcanzar la longitud máxima n , pues los eventos de copia están acotados únicamente por la cota máxima transitoria.

5.2. Tabla Comparativa de Políticas de Redimensionamiento

Cuadro 1: Comparación asintótica de políticas de crecimiento tras alcanzar longitud n .

Política	# Resizes	Total Copias	Costo Amortizado	Espacio Sin Usar
$C + 1$ (Aditiva)	$O(n)$	$O(n^2)$	$O(n)$	0 (0 %)
$2C$ (Multiplicativa)	$O(\log_2 n)$	$O(n)$	$O(1)$	n (50 %)
$1,5C$ (Multiplicativa)	$O(\log_{1,5} n)$	$O(n)$	$O(1)$	$0,5n$ (33,3 %)

5.3. Familia General de Políticas Multiplicativas $C_{\text{new}} = \lceil \alpha C \rceil$

Respuesta: NO es necesario duplicar la capacidad ($\alpha = 2$) para obtener costo amortizado constante. Cualquier política de crecimiento multiplicativo basada en un factor constante $\alpha > 1$ preserva la propiedad $O(1)$ amortizado.

5.3.1. Demostración Matemática

Sea $\alpha > 1$ el factor de crecimiento. La suma de los elementos copiados en todas las operaciones de redimensionamiento hasta alcanzar una longitud n corresponde a la suma de la serie geométrica de razón $1/\alpha < 1$:

$$\text{Total Copias} = \sum_{j=0}^k C_0 \alpha^j = C_0 \frac{\alpha^{k+1} - 1}{\alpha - 1} \leq \frac{\alpha}{\alpha - 1} \cdot n = O(n) \quad (15)$$

Dividiendo el costo total de copias entre las n operaciones de crecimiento, el costo amortizado resulta:

$$\hat{c} = \frac{O(n)}{n} = O\left(\frac{\alpha}{\alpha - 1}\right) = O(1) \quad \text{constante} \quad (16)$$

- Para $\alpha = 2$, la cota de copias es $\frac{2}{2-1} \cdot n = 2n \implies O(1)$.
- Para $\alpha = 1,5$, la cota de copias es $\frac{1,5}{1,5-1} \cdot n = 3n \implies O(1)$.
- Para políticas aditivas ($C + 1$), el factor equivale a un $\alpha \rightarrow 1$, provocando que $\frac{\alpha}{\alpha-1} \rightarrow \infty$, lo que degrada el costo amortizado a tiempo lineal $O(n)$ por operación (costo total cuadrático $O(n^2)$).